

Rapport de projet

RAG — NordTrail Gear

Retrieval Augmented Generation avec Azure OpenAI et Azure AI Search

Cours IA — UQAC

Date : 22 May 2026

Équipe

Simon Delepine (SDEL28070300)

Thomas Gourjault (GOUT24110300)

Matthis Lahargoue (LAHM11100200)

Marin Boehm (BOEM27080300)

Table des matières

- 1. Introduction
- 2. Contexte et objectifs
- 3. Architecture du système
- 4. Structure du dépôt et modules
- 5. Corpus documentaire
- 6. Pipeline d'ingestion
- 7. Pipeline de requête RAG
- 8. Configuration et déploiement
- 9. Tests et validation
- 10. Conclusion et perspectives

1. Introduction

Ce rapport présente le travail réalisé dans le cadre du cours IA à l'UQAC : la conception et l'implémentation d'un système RAG (Retrieval Augmented Generation) pour l'entreprise fictive **NordTrail Gear**, spécialisée en équipement outdoor (randonnée, trail, camping). Le système permet de répondre aux questions clients en s'appuyant sur une base documentaire interne plutôt que sur les connaissances générales du modèle de langage.

Le RAG combine deux étapes : la **récupération** de passages pertinents dans des documents indexés, puis la **génération** d'une réponse par un LLM conditionné par ce contexte. Cette approche réduit les hallucinations et garantit que les réponses reflètent les politiques réelles de l'entreprise (livraison, tailles, SAV, annulations).

2. Contexte et objectifs

2.1 Problématique métier

- Volume de courriels clients sur la livraison, les retours, les tailles et le SAV.
- Documents dispersés (FAQ, guides, procédures internes, catalogue, données JSON).
- Besoin d'un assistant capable de citer les règles applicables à chaque situation.

2.2 Objectifs techniques

- Indexer automatiquement des fichiers PDF, Markdown, CSV et JSON.
- Découper les textes en chunks adaptés à la recherche vectorielle.
- Stocker des embeddings et interroger un index par similarité sémantique.
- Produire des réponses via Azure OpenAI en injectant le contexte récupéré.
- Proposer une voie locale (ChromaDB) pour développer sans coût cloud à chaque test.

3. Architecture du système

Le projet expose **deux pipelines complémentaires** partageant le même dossier source *documents/* mais différant par le stockage vectoriel et la stratégie de chunking.

3.1 Pipeline Azure (production)

```
documents/ --> ingest_azure.py --> Azure OpenAI (embeddings)
      |
      v
    Azure AI Search (index vectoriel)
      |
      v
rag_test.py : question --> retrieve --> LLM --> réponse
```

3.2 Pipeline local Chroma (développement)

```
documents/ --> utils.py (load, clean, chunk par mots)
      |
      v
    ingest.py --> embeddings.py --> vectorstore.py (ChromaDB)
```

Différence importante : *ingest_azure.py* découpe le texte par **caractères** (CHUNK_SIZE=1500, overlap=150). Le pipeline local via *utils.chunk_text()* découpe par **mots**, ce qui préserve mieux l'intégrité des phrases lors du prototypage.

4. Structure du dépôt et modules Python

Fichier	Rôle	Fonctions clés
config.py	Configuration	Charge .env ; valide OPENAI_API_KEY et SEARCH_API_KEY ; paramètres chunk, chemins, noms
utils.py	Prétraitement	load_document_text() multi-formats ; clean_text() ; chunk_text() par mots ; list_document_files().
embeddings.py	Vectorisation	Client AzureOpenAI ; get_embedding() / get_embeddings().
vectorstore.py	Stockage local	ChromaDB persistant ; reset_collection() ; query_collection() ; cosine.
ingest.py	Ingestion locale	Orchestre utils + embeddings + Chroma ; exclut emails_clients_test.csv.
ingest_azure.py	Ingestion cloud	Extracteurs PDF/MD/CSV/JSON ; chunk caractères ; upload Azure Search.
rag_test.py	Requête RAG	VectorizedQuery ; recherche hybride ; prompt + chat completions.
test_foundry.py	Diagnostic	Test connexion Microsoft Foundry via .env.test.

4.1 config.py — Configuration centralisée

Toutes les variables sensibles proviennent d'un fichier `.env` (jamais versionné). Les paramètres couvrent Azure OpenAI (clé, endpoint, version API, noms de déploiements embedding et chat), Azure AI Search (endpoint, clé, nom d'index) et Chroma (chemin `./db/chroma_store`, collection `nordtrail_documents`). La fonction `validate_config()` interrompt l'exécution si des clés essentielles manquent.

4.2 utils.py — Chargement et découpage

- PDF : extraction page par page avec `pypdf`, préfixe [Page N].
- Markdown / TXT : lecture UTF-8 directe.
- CSV : chaque ligne convertie en texte clé:valeur lisible.
- JSON : sérialisation indentée (listes ou objet unique).
- Nettoyage : suppression `\x00`, espaces multiples, lignes vides excessives.

4.3 embeddings.py et vectorstore.py

`embeddings.py` instancie un client Azure OpenAI et appelle le déploiement configuré (ex. `non-embedding`). La version initiale traite les textes un par un pour faciliter le débogage. `vectorstore.py` encapsule ChromaDB en mode persistant avec distance cosine, adaptée aux embeddings OpenAI (typiquement 1536 dimensions).

4.4 ingest.py — Ingestion locale

Pour chaque fichier du dossier `documents/` (sauf exclusions), le script charge le texte, le nettoie, le découpe, génère les embeddings et les enregistre avec métadonnées (source, type, `chunk_index`, `chunk_id`). La collection est réinitialisée à chaque ingestion complète via `reset_collection()`. Fichier exclu explicitement : `emails_clients_test.csv`, réservé à l'évaluation et non à l'indexation.

4.5 ingest_azure.py — Ingestion cloud

- Extracteurs dédiés par extension (`.pdf`, `.md`, `.csv`, `.json`).
- Chunking par fenêtre glissante en caractères (config `CHUNK_SIZE` / `CHUNK_OVERLAP`).

- Vectorisation via Azure OpenAI puis `upload_documents()` vers l'index Azure AI Search.
- Identifiant de chunk : `{nom_fichier}_chunk_{i}` ; texte préfixé par le nom du fichier.

4.6 `rag_test.py` — Boucle RAG complète

Le script charge `.env.test`, configure les clients Search et OpenAI, puis pour chaque question : (1) vectorise la question, (2) construit une `VectorizedQuery` sur le champ vector, (3) exécute une recherche hybride (texte + vecteur), (4) assemble un prompt « réponds strictement avec le contexte », (5) appelle le modèle chat. Dix questions types couvrent livraison France, frais Belgique, colis endommagé, tailles trail, point relais, etc.

5. Corpus documentaire

Fichier	Type	Statut	Contenu principal
faq_livraison.md	Markdown	Indexé	Zones, délais, frais, colis endommagé, point relais
guide_tailles.md	Markdown	Indexé	Vestes, pantalons, chaussures trail, sacs, gants
procedure_sav_interne.md	Markdown	Indexé	Procédure SAV, escalade, traçabilité
conditions_annulation.md	Markdown	Indexé	Annulation, modification, retours
catalogue_produits.csv	CSV	Indexé	Produits, prix, tailles, garantie, notes
clients_exemples.json	JSON	Indexé	Profils clients fictifs (CL-001, etc.)
commandes_exemples.json	JSON	Indexé	Commandes, statuts, tracking, montants
emails_clients_test.csv	CSV	Exclu	14 scénarios d'évaluation RAG (intents attendus)

Les documents Markdown sont structurés (version, date, objectif) et rédigés explicitement pour alimenter le service client et l'assistant RAG. Le fichier *emails_clients_test.csv* contient pour chaque courriel : sujet, corps, intent attendu, documents attendus et résumé de réponse — il sert de jeu de test sans polluer l'index de recherche.

6. Pipeline d'ingestion — fonctionnement détaillé

6.1 Étapes communes

- Extraction du texte brut selon le format du fichier.
- Nettoyage (caractères invalides, normalisation des espaces).
- Découpage en chunks avec chevauchement pour ne pas perdre le contexte aux frontières.
- Génération d'embeddings (vecteurs numériques représentant le sens du passage).
- Stockage dans l'index avec identifiant unique et métadonnées de traçabilité.

6.2 Commande d'ingestion Azure

```
python ingest_azure.py
```

Sortie attendue : liste des fichiers traités, nombre de chunks par document, total de chunks indexés dans Azure AI Search.

6.3 Commande d'ingestion locale

```
python ingest.py
```

7. Pipeline de requête RAG

Lorsqu'un utilisateur pose une question, le système ne répond pas directement depuis le modèle. Il cherche d'abord les passages les plus proches sémantiquement dans l'index, puis construit un prompt du type :

```
Tu es un assistant basé sur des documents. Réponds strictement en utilisant le contexte
suivant.
Contexte : {passages récupérés}
Question : {question}
```

La recherche dans *rag_test.py* combine recherche textuelle et requête vectorielle (`VectorizedQuery`, `k_nearest_neighbors` configurable). Le paramètre *top* contrôle le nombre de documents retournés (souvent 1 en test, extensible à 3+).

8. Configuration et déploiement

8.1 Variables d'environnement (.env)

Variable	Description
AZURE_OPENAI_API_KEY	Clé API Azure OpenAI
AZURE_OPENAI_ENDPOINT	Endpoint régional (ex. canadacentral)
AZURE_OPENAI_EMBEDDING_MODEL	Nom du déploiement embedding
AZURE_OPENAI_CHAT_MODEL	Nom du déploiement chat
AZURE_SEARCH_API_KEY	Clé admin Azure AI Search
AZURE_SEARCH_ENDPOINT	URL du service Search
AZURE_SEARCH_INDEX_NAME	Nom de l'index (index-rag-canadien)
CHUNK_SIZE / CHUNK_OVERLAP	Paramètres de découpage (caractères, pipeline Azure)
DOCUMENTS_FOLDER	Chemin vers ./documents

8.2 Mise en route

- Copier .env.example vers .env et renseigner les clés Azure.
- Installer les dépendances : openai, azure-search-documents, chromadb, pypdf, python-dotenv, etc.
- Ingérer : `python ingest_azure.py`
- Tester : `python rag_test.py`
- Option : `python test_foundry.py` pour valider Microsoft Foundry.

La documentation `SETUP.md` décrit également les commandes Azure CLI pour récupérer endpoints et clés depuis le portail ou la ligne de commande.

9. Tests et validation

9.1 Questions intégrées dans `rag_test.py`

- Délais de livraison standard en France métropolitaine.
- Montant minimum pour livraison offerte en France.
- Procédure si colis arrivé endommagé.
- Choix de taille pour chaussures de trail (espace devant les orteils).
- Frais de livraison en Belgique, délai point relais.
- Volume de sac recommandé pour randonnée à la journée.

9.2 Jeu d'évaluation `emails_clients_test.csv`

Quatorze scénarios de courriels clients avec intents (`demande_retour`, `annulation_commande`, `modification_adresse`, `question_douane`, `demande_garantie`, `echange_taille`, etc.) et documents attendus. Ce fichier permet d'évaluer la pertinence des réponses RAG sans l'inclure dans l'index.

10. Conclusion et perspectives

10.1 Bilan

- Système RAG de bout en bout : ingestion multi-formats, index vectoriel, génération conditionnée.
- Architecture modulaire et secrets externalisés dans .env.
- Double voie Azure / Chroma pour production et prototypage.
- Corpus métier cohérent pour un cas e-commerce outdoor réaliste.

10.2 Limites actuelles

- Embeddings générés séquentiellement (lent sur gros volumes).
- top-k souvent fixé à 1 — contexte parfois insuffisant.
- Deux stratégies de chunking différentes entre pipelines local et Azure.
- Pas encore d'interface utilisateur ni de métriques automatiques sur le CSV de test.

10.3 Pistes d'amélioration

- Batch embeddings et monitoring des coûts API.
- Augmenter top-k et fusion de plusieurs chunks dans le prompt.
- Interface Streamlit ou API REST pour démonstration.
- Scoring automatique (précision intent, citation des bons documents).
- Harmonisation du chunking et métadonnées source dans l'index Azure.

Rapport généré automatiquement — Projet RAG_AzureDevOups — 2026-05-22